Connect to a Server

Explore how to establish a client-server connection using Python's requests library. Learn to send HTTP GET requests to REST API endpoints and handle responses without additional client configuration, focusing on simple and effective server communication.

REST follows the **client-server model**. The server provides endpoints to connect to and serves data as per the request. The client connects to the endpoint and requests data. The request and response always follow the HTTP request-response model. For simplicity, in place of the API endpoint, the term “service” will be used.

The task of connecting to a server can be divided into two types:

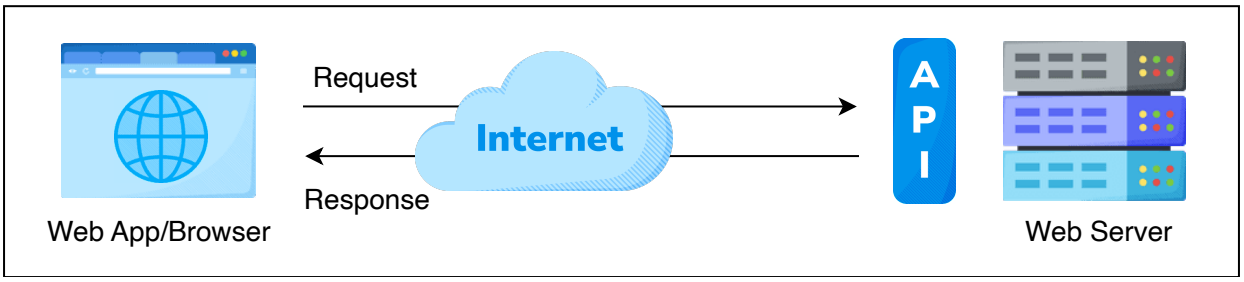
- Without configuration
- With configuration

This distinction occurs due to a variety of reasons, including but not limited to, authentication mechanisms used by the server and the certificate type of the server. In this lesson, we’ll look at how to connect to a server without the client needing to configure itself. For completeness, the client will try to fetch data, i.e., issue a GET. We’ll be connecting to the air quality service provided by MET Norway. The URI is as follows:

https://api.met.no/weatherapi/airqualityforecast/0.1/stations.

Previously, we had chosen the `requests` library to use for our client. Let’s how to use our chosen library to connect to the service.

Ask



Connecting to a server using REST API

Steps

1. Create a new file named `client.py`.
2. Import the `requests` library.

```
import requests
```

3. Create a class and name it `Client`.

```
class Client(object):
```

4. Add a parameterized constructor that accepts `url` as a parameter, and assign it to an instance variable.

```
def __init__(self, url: str):
    self.url = url
```

5. Create a method named `get` with the return type as a string.

```
def get(self) -> str:
```

6. Within the method, call the `get` method of `requests` and pass the URL to it using the instance variable `url`. Return the body of the response using the `content` attribute of the `response` object.

```
response = requests.get(self.url)
return response.content
```

Ask

7. Add a `main` block.

```
if __name__ == '__main__':
```

8. Create an instance of `Client` named `client` by passing the URI of the air quality service.

```
client = Client(
'https://api.met.no/weatherapi/airqualityforecast/0.1/stations')
```

9. Call the `get` method of the `client` and print the result.

```
print(client.get())
```

Show Result

How it works

The core of the code is in the `get` method of the `Client` class. The `get` method calls the `get` method of `requests`. As mentioned earlier, the `requests` library provides wrappers to make REST calls easier. The `get` method is one such wrapper. It encapsulates the lower-level details of connecting to the service. The parameter is the URL of the service, which, in this case, is provided by the instance variable `url`.

Ask

Request-response cycle for connecting to a server

The response from the service is represented by the `response` object, which is returned by the `get` method. The `response` object contains, at the minimum, the status code returned by the server and the content if the call has been successful. If the call results in an error, the status code will contain the HTTP error code, and the content might contain the error details. The reason behind saying “might” is that some services just return the error code and no details of the error; other services may provide a link to know more about the error.

As might be guessed, the `content` attribute of the `response` object contai
content returned by the service. The `get` method returns the content as its result, which is then printed by the `main` block.

Ask

Try it yourself

- Follow the steps above to complete the code for this lesson.
- Click “Run” to execute the code and display the result.

client.py Python 3.8

```
### Write the code here
#
#
```

Run

When the code is executed, we'll get station details from MET Norway

Solution

If stuck, refer to the final code below for some guidance.

Please note that the file below is read-only.

client.py Python 3.8

Ask

```
import requests


class Client(object):
```

```
    def __init__(self, url: str):
        self.url = url

    def get(self) -> str:
        return requests.get(self.url).content
```

Ask